เอกสารประกอบการเรียนปฏิบัติการครั้งที่ 3

เรื่อง **การสร้างฐานข้อมูล การสร้าง Index การสร้าง View และ การใช้คำสั่ง Select**

วัตถุประสงค์ : เมื่อจบบทปฏิบัติการนี้แล้วนักศึกษาสามารถ

1. สามารถสร้างฐานข้อมูลและตารางได้
2. สามารถใช้คำสั่ง Select เพื่อเลือกแสดงข้อมูลได้
3. สามารถสร้างอินเด็กซ์ (Index) แบบ Clustered Index และ Non Clustered Index ได้
4. สามารถสร้างและจัดการวิว (View) ได้

ขั้นตอนการปฏิบัติ (4 ชั่วโมง)

ส่วนที่ 1

1. สร้างฐานข้อมูล (Database) และตาราง (Table) โดยใช้คำสั่ง SQL (40 นาที)
2. กำหนดค่า Primary Key Constraints และ Foreign Key Constraints โดย ใช้คำสั่ง SQL (20 นาที)
3. INSERT ข้อมูลลงในตารางโดยใช้คำสั่ง SQL (60 นาที)

ส่วนที่ 2

1. สร้าง แก้ไข และลบอินเด็กซ์ (Index) (30 นาที)

ส่วนที่ 3

1. สร้าง แก้ไข และลบวิว (View) (30 นาที)

ส่วนที่ 4

2. ศึกษาและใช้คำสั่ง Select เพื่อแสดงข้อมูล (60 นาที)

1101171 การออกแบบและพัฒนาฐานข้อมูล

Database Design and Development

สำนักวิชาศาสตร์และศิลป์ดิจิทัล มหาวิทยาลัยเทคโนโลยีสุรนารี

ผู้ช่วยศาสตราจารย์ ดร.สถิตย์โชค โพธิ์สอาด

นางสาวสุภลักษณ์ สีสุกอง

นางสาววฏ กาญจนพฤฒิพงศ์

<u>ส่วนที่ 1</u> : การสร้างฐานข้อมูล (Database) และ ตาราง (Table)

1. สร้างโฟลเดอร์เป็นรหัสนักศึกษาเก็บไว้ที่ไดร์ฟ C
2. สร้างฐานข้อมูลชื่อ dbemployeeXXXX โดยกำหนดให้ XXXX คือ เลข 4 ตัวท้ายของรหัสนักศึกษา และเก็บไว้ที่โฟลเดอร์ที่สร้างไว้ เช่น C:\B65XXXXX\dbemployeeXXXX ซึ่งภายในฐานข้อมูลประกอบด้วย 4 ตาราง และมีข้อมูลในแต่ละตารางดังนี้

ตาราง EMPLOYEE

Field Name	Data Type	Field Size
empnum (PK)	char	4
empname	varchar	15
hiredate	date/time	
salary	int	
position	varchar	15
depno (FK)	char	2
mgrno	char	4

ตาราง PROJECT

Field Name	Data Type	Field Size
projno (PK)	char	2
projdesc	varchar	20
startdate	date/time	
enddate	date/time	
budget	int	

ตาราง DEP

Field Name	Data Type	Field Size
depno (PK)	char	2
depname	varchar	15
location	varchar	15

ตาราง PROJWORK

Field Name	Data Type	Field Size
projno (FK)	char	2
empnum (FK)	char	4
hours	int	

ข้อมูลในตาราง EMPLOYEE

empnum	empname	hiredate	salary	position	depno	mgrno
1001	Anya	1992-06-12	15000	clerk	10	1002
1002	Arnon	1991-03-15	25000	clerk	10	1003
1003	Chana	1987-07-08	60000	manager	10	2002
2001	Darin	1997-07-20	27000	clerk	20	1002
2002	Gavin	1987-10-10	55000	manager	20	2003
2003	Kris	2002-01-01	30000	clerk	20	
3001	Sandy	1980-05-05	70000	supervisor	30	2002
3002	Tara	2001-02-01	22000	clerk	30	3004
3003	Tisha	1983-09-10	90000	supervisor	30	3004
xxxx (เลข 4 ตัวท้ายของรหัสนักศึกษา)	Studentname (ใส่ชื่อนักศึกษา)	1990-08-16	50000	manager	40	2002

ข้อมูลในตาราง DEP

depno	depname	location
10	accounting	silom
20	administration	sukumvit
30	marketing	ratchada
40	finance	silom
50	research	sukumvit

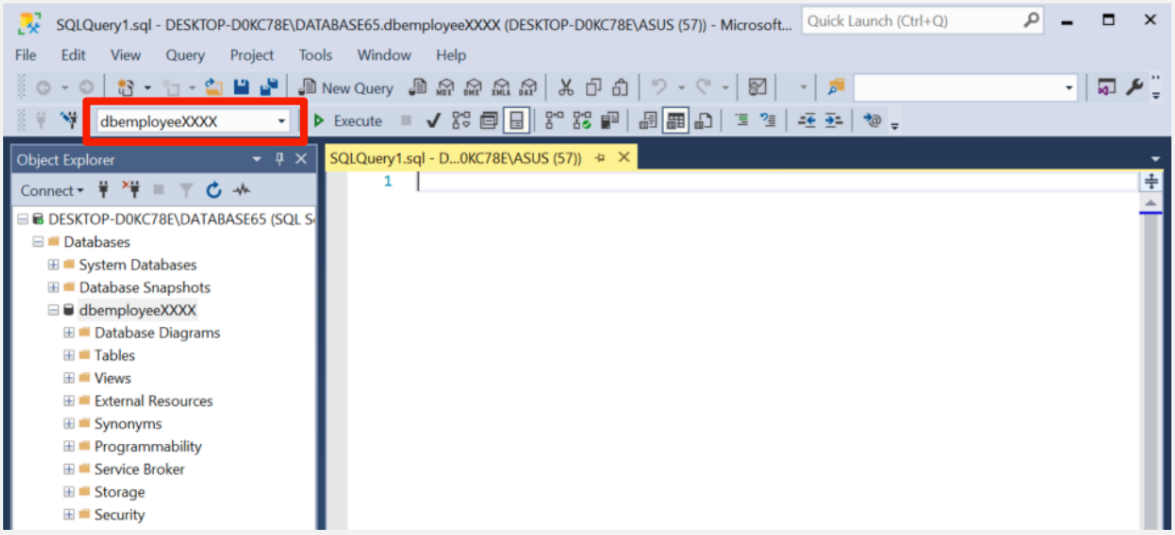
ข้อมูลในตาราง PROJECT

projno	projdesc	startdate	enddate	budget
01	new project	2022-10-10	2023-10-10	1000000
02	bill collection	2022-05-09	2023-09-20	5000000
03	new building	2022-03-08	2024-08-30	70000000

ข้อมูลในตาราง PROJWORK

projno	empnum	hours
01	3001	25
01	3003	30
02	XXXX (เลข 4 ตัวท้ายของรหัสนักศึกษา)	40
03	2003	20
03	2001	30

3. เมื่อสร้างฐานข้อมูลเรียบร้อยให้ Refresh หรือกด F5 และเปลี่ยนชื่อฐานข้อมูลจาก master เป็นชื่อฐานข้อมูลที่ได้สร้างไว้ คือ dbemployeeXXXX



4. สร้างตารางและกำหนด Primary Key ให้กับแต่ละตารางโดยใช้คำสั่งดังนี้

ชื่อตาราง	คำสั่ง

Employee	`CREATE TABLE employee`
	`(`
	`empnum char(4),`
	`empname varchar(15),`
	`hiredate datetime,`
	`salary int,`
	`position varchar(15),`
	`depno char(2),`
	`mgrno char(4),`
	`CONSTRAINT employee_PK PRIMARY KEY (empnum)`
	`)`

DEP	`CREATE TABLE dep`
	`(`
	`depno char(2),`
	`depname varchar(15),`
	`location varchar(15),`
	`CONSTRAINT depno_PK PRIMARY KEY (depno)`
	`)`

PROJECT	`CREATE TABLE project`
	`(`
	`projno char(2),`
	`projdesc varchar(20),`
	`startdate datetime,`
	`enddate datetime,`
	`budget int,`
	`CONSTRAINT projno_PK PRIMARY KEY (projno)`
	`)`

PROJWORK	`CREATE TABLE projwork (`
	`projno char(2),`
	`empnum char(4),`
	`hours int`
	`)`

5. กำหนด Foreign Key ให้กับแต่ละตารางโดยใช้คำสั่งดังนี้

กำหนด Foreign Key ให้กับ ตาราง EMPLOYEE

```
ALTER TABLE employee
ADD CONSTRAINT depno_fk FOREIGN KEY (depno) REFERENCES dep
(depno)
```

กำหนด Foreign Key ให้กับ ตาราง PROJWORK

```
ALTER TABLE projwork
ADD CONSTRAINT projno_fk FOREIGN KEY (projno) REFERENCES
project (projno)

ALTER TABLE projwork
ADD CONSTRAINT empnum_fk FOREIGN KEY (empnum) REFERENCES
employee (empnum)
```

6. กรอกข้อมูลลงในตารางโดยใช้คำสั่งดังนี้

ชื่อตาราง	คำสั่ง
DEP	INSERT INTO dep (depno,depname,location) VALUES('10','accounting','silom'); INSERT INTO dep (depno,depname,location) VALUES('20','administration','sukumvit'); INSERT INTO dep (depno,depname,location) VALUES('30','marketing','ratchada'); INSERT INTO dep (depno,depname,location) VALUES('40','finance','silom') INSERT INTO dep (depno,depname,location) VALUES('50','research','sukumvit')
PROJECT	INSERT INTO project(projno,projdesc,startdate,enddate,budget) VALUES ('01','new project','2022-10-10','2023-10-10','1000000') INSERT INTO project(projno,projdesc,startdate,enddate,budget) VALUES ('02','bill collection','2022-05-09','2023-09-20','5000000') INSERT INTO project(projno,projdesc,startdate,enddate,budget) VALUES ('03','new building','2022-03-08','2024-08-30','70000000')
EMPLOYEE	INSERT INTO employee(empnum,empname,hiredate,salary,position,depno,mgrno) VALUES ('1001','Anya','1992-06-12','15000','clerk','10','1002') INSERT INTO employee(empnum,empname,hiredate,salary,position,depno,mgrno) VALUES ('1002','Arnon','1991-03-15','25000','clerk','10','1003')

	INSERT INTO employee(empnum,empname,hiredate,salary,position,depno,mgrno) VALUES ('1003','Chana','1987-07-08','60000','manager','10','2002') INSERT INTO employee(empnum,empname,hiredate,salary,position,depno,mgrno) VALUES ('2001','Darin','1997-07-20','27000','clerk','20','1002') INSERT INTO employee(empnum,empname,hiredate,salary,position,depno,mgrno) VALUES ('2002','Gavin','1987-10-10','55000','manager','20','2003') INSERT INTO employee(empnum,empname,hiredate,salary,position,depno,mgrno) VALUES ('2003','Kris','2002-01-01','30000','clerk','20','') INSERT INTO employee(empnum,empname,hiredate,salary,position,depno,mgrno) VALUES ('3001','Sandy','1982-05-05','70000','supervisor','30','2002') INSERT INTO employee(empnum,empname,hiredate,salary,position,depno,mgrno) VALUES ('3002','Tara','2001-02-01','22000','clerk','30','3004') INSERT INTO employee(empnum,empname,hiredate,salary,position,depno,mgrno) VALUES ('3003','Tisha','1983-09-10','90000','supervisor','30','3004') INSERT INTO employee(empnum,empname,hiredate,salary,position,depno,mgrno) VALUES ('Last 4 digits of student ID','Studentname','1990-08-16','50000','manager','40','2002')
PROJWORK	INSERT INTO projwork(projno,empnum,hours) VALUES('01','3001','25') INSERT INTO projwork(projno,empnum,hours) VALUES('01','3003','30') INSERT INTO projwork(projno,empnum,hours) VALUES('02','4001','40') INSERT INTO projwork(projno,empnum,hours) VALUES('03','2003','20') INSERT INTO projwork(projno,empnum,hours) VALUES('03','2001','30')

<u>ส่วนที่ 2</u> : การสร้างอินเด็กซ์ (Index)

การสร้างอินเด็กซ์มีวัตถุประสงค์หลักเพื่อให้การค้นหาข้อมูลรวดเร็วขึ้น โดยเฉพาะอย่างยิ่งในกรณีที่ข้อมูลมีจำนวนมาก รูปแบบของอินเด็กซ์ในโปรแกรม Microsoft SQL Server มี 2 ประเภท คือ Clustered Index และ Nonclustered Index

การสร้างอินเด็กซ์ สามารถทำได้ 2 วิธีดังนี้

1. การสร้างอินเด็กซ์พร้อมกับการสร้างตาราง
2. การสร้างอินเด็กซ์หลังจากการสร้างตาราง

1. การสร้างอินเด็กซ์พร้อมกับการสร้างตาราง

คำสั่งต่อไปนี้เป็นการสร้างตาราง PERSON1 พร้อมกับการสร้างอินเด็กซ์ โดยกำหนดให้ personid ซึ่งเป็น Primary Key เป็นอินเด็กซ์แบบ NONCLUSTERED และกำหนดให้ socialsecuritynumber เป็นอินเด็กซ์แบบ CLUSTERED พร้อมทั้งกำหนด CONSTRAINT เป็น UNIQUE ทั้งนี้ การสร้างอินเด็กซ์พร้อมกับการสร้างตารางข้อมูลจะกระทำได้กับเฉพาะฟิลด์ที่กำหนดเป็น Primary Key หรือ UNIQUE เท่านั้น

```
1  CREATE TABLE person1
2  (
3  personid smallint PRIMARY KEY NONCLUSTERED,
4  firstname varchar(30),
5  lastname varchar(30),
6  socialsecuritynumber char(11) UNIQUE CLUSTERED
7  )
```

จากนั้นให้ใช้คำสั่งในการแสดงข้อมูลของอินเด็กซ์ ดังนี้

```
sp_helpindex person1
```

จากนั้นจะปรากฏหน้าจอในการแสดงข้อมูลของอินเด็กซ์ ดังนี้

Results | Messages

	index_name	index_description	index_keys
1	PK__person1__EC7C6174C51E463C	nonclustered, unique, primary key located on PRIM...	personid
2	UQ__person1__4E18FB6930E5DD3B	clustered, unique, unique key located on PRIMARY	socialsecuritynumber

คำสั่งต่อไปนี้เป็นตัวอย่างการสร้างตาราง PERSON2 พร้อมกับการสร้างอินเด็กซ์ทั้งแบบ CLUSTERED และ NONCLUSTERED

```
1  CREATE TABLE person2
2  (
3  personid smallint PRIMARY KEY CLUSTERED,
4  firstname varchar(30),
5  lastname varchar(30),
6  socialsecuritynumber char(11) UNIQUE NONCLUSTERED
7  )
```

จากนั้นจะปรากฏหน้าจอในการแสดงข้อมูลของอินเด็กซ์ ดังนี้

Results | Messages

	index_name	index_description	index_keys
1	PK__person2__EC7C6175EC587C8F	clustered, unique, primary key located on PRIMARY	personid
2	UQ__person2__4E18FB68ADD72600	nonclustered, unique, unique key located on PRIMA...	socialsecuritynumber

2. การสร้างอินเด็กซ์หลังการสร้างตารางข้อมูล

สามารถทำได้โดยการใช้คำสั่ง CREATE INDEX ดังตัวอย่างต่อไปนี้ เป็นการสร้างอินเด็กซ์แบบ NONCLUSTERED สำหรับฟิลด์ firstName

```
9  CREATE INDEX INDEX_firstname
10 ON person2(firstname)
```

จากนั้นจะปรากฏหน้าจอในการแสดงข้อมูลของอินเด็กซ์ ดังนี้

Results | Messages

	index_name	index_description	index_keys
1	INDEX_firstname	nonclustered located on PRIMARY	firstname
2	PK__person2__EC7C6175EC587C8F	clustered, unique, primary key located on PRIMARY	personid
3	UQ__person2__4E18FB68ADD72600	nonclustered, unique, unique key located on PRIMA...	socialsecuritynumber

ให้สร้างอินเด็กซ์ดังคำสั่งต่อไปนี้

```
12  CREATE UNIQUE NONCLUSTERED INDEX INDEX_firstname
13  ON person2 (firstname,lastname DESC)
14  WITH FILLFACTOR = 60,
15  IGNORE_DUP_KEY,
16  DROP_EXISTING,
17  SORT_IN_TEMPDB
```

ความหมายของแต่ละคำสั่งย่อย มีดังนี้

- ♥ CREATE UNIQUE NONCLUSTERED INDEX เป็นการสร้างอินเด็กซ์แบบ NONCLUSTERED และแบบ UNIQUE โดยการนำเอาฟิลด์ firstname และ lastname มารวมกันโดยเรียงลาดับจากมากไปหาน้อย (DESC)
- ♥ WITH FILLFACTOR = 60 เป็นการสร้างโดยใช้พื้นที่เพียง 60% เหลืออีก 40% สำหรับเป็นพื้นที่ว่างเพื่อรองรับการเพิ่มข้อมูลในอนาคต
- ♥ IGNORE_DUP_KEY เป็นการยกเลิกการเพิ่มข้อมูลถ้ามีข้อมูลซ้ำในอินเด็กซ์
- ♥ DROP_EXISTING เป็นการลบอินเด็กซ์ที่มีอยู่ออกแล้วสร้างใหม่ในชื่อเดิม
- ♥ SORT_IN_TEMPDB ให้ทำการเรียงลาดับข้อมูลในฐานข้อมูล TEMPDB เพื่อให้ทำงานได้เร็วขึ้น

จากนั้นจะปรากฏหน้าจอในการแสดงข้อมูลของอินเด็กซ์ ดังนี้

Results | Messages

	index_name	index_description	index_keys
1	INDEX_firstname	nonclustered, ignore duplicate keys, unique located ...	firstname, lastname(-)
2	PK__person2__EC7C6175EC587C8F	clustered, unique, primary key located on PRIMARY	personid
3	UQ__person2__4E18FB68ADD72600	nonclustered, unique, unique key located on PRIMA...	socialsecuritynumber

การ Rebuild อินเด็กซ์

เป็นการเปลี่ยนแปลงโครงสร้างบางอย่างแล้วให้สร้างอินเด็กซ์ใหม่ โดยใช้คำสั่งดังตัวอย่างต่อไปนี้

```
DBCC DBREINDEX (person2,INDEX_firstname,70)
```

คำสั่งนี้เป็นการ rebuild อินเด็กซ์ชื่อ INDEX_firstName โดยกำหนดให้ FILLFACTOR เท่ากับ 70%

การเปลี่ยนชื่ออินเด็กซ์

```
sp_rename
@objname = 'person2.INDEX_firstname',
@newname = 'INDEX_first_lastname',
@objtype = 'INDEX'
```

ใช้คำสั่ง sp_helpindex person2 ตรวจสอบการเปลี่ยนชื่ออินเด็กซ์

Results | Messages

	index_name	index_description	index_keys
1	INDEX_first_lastname	nonclustered, ignore duplicate keys, unique located ...	firstname, lastname(-)
2	PK__person2__EC7C6175EC587C8F	clustered, unique, primary key located on PRIMARY	personid
3	UQ__person2__4E18FB68ADD72600	nonclustered, unique, unique key located on PRIMA...	socialsecuritynumber

การลบอินเด็กซ์

```
DROP INDEX person2.INDEX_first_lastname
```

ใช้คำสั่ง sp_helpindex person2 ตรวจสอบการเปลี่ยนชื่ออินเด็กซ์

Results | Messages

	index_name	index_description	index_keys
1	PK__person2__EC7C6175EC587C8F	clustered, unique, primary key located on PRIMARY	personid
2	UQ__person2__4E18FB68ADD72600	nonclustered, unique, unique key located on PRIMA...	socialsecuritynumber

<u>ส่วนที่ 3</u> : การสร้างวิว (View)

วิว (View) เปรียบเสมือนหน้าต่างของข้อมูลจากตารางหนึ่ง ๆ ที่ถูกเรียกเพื่อทำการเปลี่ยนแปลงข้อมูล วิวเป็นตารางเสมือนจริง (Virtual Table) ที่มีข้อมูลมาจากตารางหลัก (Base Table) โดยไม่มีข้อมูลเก็บอยู่จริง

ประโยชน์ของวิว คือ ผู้ใช้ในระบบสามารถเรียกดูข้อมูลจากฐานข้อมูลในลักษณะที่ผู้ใช้นั้น ๆ ต้องการ โดยใช้คำสั่งง่าย ๆ ซึ่งอาจจะเป็นการเรียกดูข้อมูลจากตารางเดียวหรือหลายตารางก็ได้ นอกจากนี้วิวยังทำให้ผู้ใช้ข้อมูลมีความเป็นอิสระในการเรียกใช้ข้อมูลหรือแก้ไขข้อมูลจากวิวที่ผู้ใช้นั้น ๆ สร้างขึ้น ทำได้โดยการใช้คำสั่ง CREATE VIEW ดังตัวอย่างต่อไปนี้

การสร้างวิวชื่อ EMPVIEW ซึ่งมีข้อมูลรหัสพนักงาน ชื่อ วันที่เริ่มทำงาน และเงินเดือน

```
1  CREATE VIEW EMPVIEW
2  AS
3  SELECT empnum,empname,hiredate,salary
4  FROM employee
```

จากนั้นให้ทำการเรียกดูข้อมูลจาก EMPVIEW โดยใช้คำสั่ง `SELECT * FROM EMPVIEW` จะปรากฏข้อมูลดังนี้

Results | Messages

	empnum	empname	hiredate	salary
1	1001	Anya	1992-06-12 00:00:00.000	15000
2	1002	Arnon	1991-03-15 00:00:00.000	25000
3	1003	Chana	1987-07-08 00:00:00.000	60000
4	2001	Darin	1997-07-20 00:00:00.000	27000
5	2002	Gavin	1987-10-10 00:00:00.000	55000
6	2003	Kris	2002-01-01 00:00:00.000	30000
7	3001	Sandy	1982-05-05 00:00:00.000	70000
8	3002	Tara	2001-02-01 00:00:00.000	22000
9	3003	Tisha	1983-09-10 00:00:00.000	90000
10	4001	Studentname	1990-08-16 00:00:00.000	50000

การลบวิว

ผู้ที่สร้างวิวขึ้นมาจะทำการลบวิวที่สร้างขึ้นได้ตามความต้องการ โดยการลบวิวจะไม่กระทบถึงตารางหลักที่วิวนั้นใช้ในการสร้างวิวขึ้นมา

```
DROP VIEW EMPVIEW
```

การใช้ WITH CHECK OPTION

คำสั่งต่อไปนี้เป็นการสร้างวิวชื่อ EMPDEP ประกอบด้วย ข้อมูลรหัสพนักงาน ชื่อพนักงานที่มีรหัสแผนก 20 โดยกำหนดให้มี WITH CHECK OPTION ซึ่งผลของคำสั่งนี้ หากมีการเพิ่มเติม แก้ไข หรือลบข้อมูลของรหัสแผนกที่ไม่ใช่ 20 ตามที่ระบุไว้ที่อนุประโยค WHERE เครื่องจะแสดงข้อความว่าไม่ถูกต้องทันที

```
1  CREATE VIEW EMPDEP
2  AS
3  SELECT empnum,empname
4  FROM employee
5  WHERE depno = '20'
6  WITH CHECK OPTION
```

การปรับปรุงแก้ไขข้อมูลในวิว

การปรับปรุงแก้ไขข้อมูลในวิวสามารถทำได้โดยใช้คำสั่ง INSERT UPDATE หรือ DELETE ทำนองเดียวกับที่ใช้ในการปรับปรุงแก้ไขข้อมูลในตารางหลัก แต่การปรับปรุงแก้ไขข้อมูลในวิวนั้นจะไม่สามารถทำได้ถ้าวิวนั้น ๆ มีข้อมูลที่เกิดจากการสร้างวิวที่มีคุณสมบัติในข้อใดข้อหนึ่งต่อไปนี้

คุณสมบัติของวิวที่ไม่สามารถปรับปรุงได้	DELETE	UPDATE	INSERT
1. ข้อมูลในวิวนั้นเกิดจากการเชื่อมโยงข้อมูล (JOIN)	✓	✓	✓
2. มีการใช้ฟังก์ชันเกี่ยวข้องกับการรวม (Group Functions)	✓	✓	✓
3. มีการใช้อนุประโยค GROUP BY	✓	✓	✓
4. มีการใช้ DISTINCT	✓	✓	✓
5. มีการใช้ข้อคำถามย่อย (Subquery)	✓	✓	✓
6. คอลัมน์บางคอลัมน์ในวิวถูกกำหนดขึ้นในลักษณะของ นิพจน์ที่เป็นการคำนวณทางคณิตศาสตร์		✓	
7. คอลัมน์ในตารางหลักที่จะต้องไม่เป็นค่าว่าง (NOT NULL) ไม่ได้ถูกรวมอยู่ในวิว			✓

โจทย์ปฏิบัติการที่ 1 (1 ลายเซ็น)

ให้สร้างวิวชื่อ DEPSAL ซึ่งประกอบด้วยข้อมูลรหัสแผนก ชื่อแผนก และที่อยู่ จากตาราง DEP โดยแสดงเฉพาะข้อมูลที่มีชื่อแผนก คือ finace หรือที่อยู่ คือ silom

ตัวอย่างผลลัพธ์ :

Results | Messages

	depno	depname	location
1	10	accounting	silom
2	40	finance	silom

โจทย์ปฏิบัติการที่ 2 (1 ลายเซ็น)

ให้สร้างวิวชื่อ EMPCHECK ซึ่งประกอบด้วยข้อมูลรหัสพนักงาน ชื่อพนักงาน ตำแหน่งงาน และรหัสแผนกของพนักงาน และมีอนุประโยค WITH CHECK OPTION เพื่อตรวจสอบว่าการใส่รหัสแผนกใน ตาราง EMPLOYEE เป็นรหัสที่มีอยู่ในตาราง DEP เท่านั้น

ตัวอย่างผลลัพธ์ :

Results | Messages

	empnum	empname	position	depno
1	1001	Anya	clerk	10
2	1002	Arnon	clerk	10
3	1003	Chana	manager	10
4	2001	Darin	clerk	20
5	2002	Gavin	manager	20
6	2003	Kris	clerk	20
7	3001	Sandy	supervisor	30
8	3002	Tara	clerk	30
9	3003	Tisha	supervisor	30
10	4001	Studentname	manager	40

<u>ส่วนที่ 4</u> : การใช้คำสั่ง Select เพื่อแสดงข้อมูล

1. **การใช้ Select**

1.1. การใช้ Select แสดงข้อมูลทั้งหมดในตาราง

```
SELECT * FROM employee
```

1.2. การใช้ SELECT แสดงข้อมูลบางคอลัมน์

```
SELECT empname, position, salary
FROM employee
```

2. **การใช้ ORDER BY แสดงข้อมูลโดยการเรียงลำดับ**

2.1. แสดงข้อมูลโดยการเรียงลำดับจากน้อยไปหามาก (โดยปกติจะเรียกจากน้อยไปหามากอยู่แล้ว เป็นค่า default ไม่ต้องระบุก็ได้)

```
SELECT * FROM projwork
ORDER BY hours ASC
```

2.2. การแสดงข้อมูลโดยการเรียงลำดับจากมากไปหาน้อย

```
SELECT * FROM projwork
ORDER BY hours DESC
```

3. **การใช้ DISTINCT**

3.1. การแสดงข้อมูลที่ซ้ำกัน โดยให้แสดงเพียงค่าละ 1 ครั้ง

```
SELECT DISTINCT projno
FROM projwork
```

4. **การใช้ TOP n**

4.1. การแสดงข้อมูลเพียง 3 ลำดับแรกเท่านั้น

```
SELECT TOP 3 empname, salary
FROM employee
ORDER BY salary
```

5. การใช้ INTO

5.1. การสร้างตารางใหม่ชื่อ emp1

```
SELECT empname, salary, position
INTO emp1
FROM employee
ORDER BY empname
```

6. การใช้ AS

6.1. การใช้ AS แสดงชื่อคอลัมน์ใหม่ตามชื่อที่กำหนดหลัง AS

```
SELECT empname AS 'employee name', depno AS
'department'
FROM employee
ORDER BY empname
```

6.2. การใช้ AS แสดงชื่อรีเลชั่น โดยใช้ชื่อตารางที่กำหนดไว้หลัง AS แทนในคำสั่ง จะทำให้คำสั่งสั้นลง

```
SELECT e1.empname, e1.position
FROM employee AS e1
ORDER BY empname
```

7. การใช้ WHERE

7.1. การใช้ WHERE กับเครื่องหมายเปรียบเทียบ =, >, >=, <, <= <>

<u>ตัวอย่างที่ 1</u> การใช้ WHERE แสดงเฉพาะผู้ที่มีตำแหน่ง manager เท่านั้น

```
SELECT empname, hiredate
FROM employee
WHERE position = 'manager'
```

<u>ตัวอย่างที่ 2</u> การใช้ WHERE แสดงเฉพาะผู้ที่มีเงินเดือนอยู่ระหว่าง 10000 ถึง 30000 เท่านั้น

```
SELECT empnum ,empname, position, salary
FROM employee
WHERE salary BETWEEN 10000 AND 30000
```

7.2. การใช้ IN แสดงข้อมูลเฉพาะพนักงานที่มีรหัส 1002, 3003, 4001

```
SELECT empnum ,empname, position
FROM employee
WHERE empnum IN ('1002','3003','4001')
```

7.3. การใช้ LIKE

<u>ตัวอย่างที่ 1</u> การใช้ LIKE แสดงข้อมูลเฉพาะพนักงานที่มีชื่อขึ้นต้นด้วยตัวอักษร A สำหรับเครื่องหมาย % ใช้แทนตัวอักษรใดก็ได้

```
SELECT empnum ,empname, position
FROM employee
WHERE empname LIKE 'A%'
```

<u>ตัวอย่างที่ 2</u> การใช้ LIKE แสดงข้อมูลเฉพาะพนักงานที่มีชื่อ 6 ตัวอักษร สำหรับเครื่องหมาย _ ใช้แทนตัวอักษรใดก็ได้ จำนวน 1 ตัว

```
SELECT empname
FROM employee
WHERE empname LIKE '______'
```

<u>ตัวอย่างที่ 3</u> การใช้ LIKE แสดงข้อมูลพนักงานที่มีตัวอักษร T อยู่ในชื่อ

```
SELECT empname
FROM employee
WHERE empname LIKE '%T%'
```

7.4. การใช้ NOT แสดงข้อมูลพนักงานที่ไม่มีตัวอักษร A อยู่ในชื่อ

```
SELECT empname
FROM employee
WHERE empname NOT LIKE '%A%'
```

7.5. การใช้ AND แสดงข้อมูลพนักงานที่มีเงินเดือนมากกว่า 20000 และมีวันเริ่มทำงานตั้งแต่ 01/01/1994

```
SELECT empname, salary, hiredate
FROM employee
WHERE salary > 20000 AND hiredate >='01/01/1994'
```

7.6. การใช้ OR แสดงข้อมูลพนักงานที่มีเงินเดือนมากกว่า 20000 หรือมีวันเริ่มทำงานตั้งแต่ 01/01/1994

```
SELECT empname, salary, hiredate
FROM employee
WHERE salary > 20000 OR hiredate >='01/01/1994'
```

โจทย์ปฏิบัติการที่ 3 (1 ลายเซ็น)

ให้แสดงข้อมูลพนักงานที่มีตัวอักษร N อยู่ในชื่อ และตำแหน่งงานเป็น clerk โดยเรียงลำดับเงินเดือนจากมากไปหาน้อย

ตัวอย่างผลลัพธ์ :

Results | Messages

	empname	salary	position
1	Darin	27000	clerk
2	Arnon	25000	clerk
3	Anya	15000	clerk

โจทย์ปฏิบัติการที่ 4 (1 ลายเซ็น)

ให้รวมข้อมูลของคอลัมน์ชื่อพนักงานและตำแหน่งงานเป็นคอลัมน์ใหม่ ชื่อ employee name โดยเรียงลำดับรหัสพนักงานจากมากไปน้อย และให้แสดงเฉพาะ 5 อันดับแรกเท่านั้น

ตัวอย่างผลลัพธ์ :

Results | Messages

	empnum	employee name
1	4001	Studentname manager
2	3003	Tisha supervisor
3	3002	Tara clerk
4	3001	Sandy supervisor
5	2003	Kris clerk